

MADRE

Agentic
System

• MODEL-AGNOSTIC • DELAYED • REASONING • EFFORT • [Beta v0.1]

Project Definition and Architecture Documentation

Scvria Software
May 24, 2026

Contents

Front Matter	1
1.1 Executive Definition	1
1.2 Why MADRE Agentic System?	1
Research Foundation	7
2.1 Research Problem	7
2.2 Research Questions	7
2.3 Hypotheses	8
2.4 Expected Contributions	8
2.5 Novelty And Validation Strategy	8
2.6 Related Work And Evidence Map	9
Product Definition	10
3.1 Executive Definition	10
3.2 Product Rationale	10
3.3 Product Design	11
3.3.1 Product Scope	11
3.3.2 What MADRE Is	11
3.3.3 What MADRE Is Not	11
3.3.4 Core User Value	11
3.3.5 Design Principles	12
3.3.6 Product Boundaries	12
Operational Scenarios	13
4.1 Foreground Lane Continuity	13
4.2 Delayed Research	13
4.3 Knowledge Extraction	13
4.4 Specialized Conversation Mutation	13
4.5 Workflow Creation	13
4.6 Developer Extension	13
Conceptual Architecture	15
5.1 Formal Requirements	15
5.2 High-Level Architecture	15
5.2.1 Conceptual Components	16
5.2.2 Responsibility Layer Diagram	17
5.2.3 Concurrent Agent Lanes	17
5.3 Runtime Data Flow	17

5.3.1	Canonical Request Narrative	18
5.3.2	Governed Model And Action Sequence	18
5.3.3	Data And Authority Rules	21
5.3.4	Trust Boundary Diagram	22
5.3.5	Access Surfaces	22
Safety And Evolution Policy		24
6.1	Runtime Safety	24
6.1.1	Policy Before Action	24
6.1.2	Autonomy Levels	24
6.2	Evolution Control	24
6.2.1	Evolution Rules	24
System Engineering Baseline		26
7.1	Engineering Problem	26
7.2	Stakeholders And Concerns	26
7.3	Requirement Domains	26
7.4	Traceability Surface	27
7.5	Quality Attributes	27
7.6	Risk And Policy Boundaries	27
Requirements Baseline		28
8.1	Requirement Taxonomy	28
8.2	Functional Requirements	29
8.3	Cross-Cutting Requirements	30
Traceability Model		32
9.1	Traceability Method	32
9.2	Architecture Traceability Matrix	32
9.3	Integrated Architecture Trace	33
Design Decision Traceability		35
Architecture Viewpoints		37
11.1	Viewpoint Catalogue	37
11.2	Viewpoint Design Focus	38
Runtime Governance		40
12.1	Runtime Authority Rule	40
12.2	Policy And Action Gate	40
12.3	Policy Matrix	40
12.4	Learning Lifecycle	41
Threat Model		42
13.1	Threat Taxonomy	42
Quality Model		44
14.1	Quality Attribute Scenarios	44

UX, Observability, And Recovery	46
15.1 Interaction Contract	46
15.2 Observability Surfaces	46
Evaluation And Benchmark Specification	47
16.1 Benchmark Families	47
Governance, Deployment, And Limits	49
17.1 Secure Engineering Principles	49
17.2 Open-Source Governance	49
17.3 Deployment And Operational Assumptions	49
17.4 Risks, Limitations, And Excluded Functions	49
Requirements Catalogue	50
Traceability Matrix	51
Threat Catalogue	52
Benchmark Catalogue	53
Conceptual Module And Agent Sketch	54
E.1 Module Ecosystem Sketch	54
E.2 Agent, Workflow, Routine, And Plan Sketch	55
E.3 Candidate And Artifact Sketch	55
Glossary	56
Product Boundary Statement	58

Front Matter

Abstract

1.1 Executive Definition

"MADRE is a local-first, model-agnostic runtime kernel for building agentic systems that preserve user sovereignty while coordinating Complex Agents, module-local reasoning lanes, governed context, internal actions, memory, learning, recovery, and inspectable execution."

"

MADRE Agentic System[Beta v0.1] exists because the current AI market is normalizing a dangerous bargain: users receive convenient language interfaces, while their data, dependency chain, costs, and decision power move into remote platforms they do not control. That bargain is technically unnecessary and politically unacceptable for many individuals, teams, public-interest organizations, researchers, creators, and small developers.

The project does not treat a language model as a complete intelligence. A model is a powerful linguistic function: useful, fallible, statistically trained, expensive in context, and unsafe when it is allowed to become the whole system. **MADRE** puts that function inside an engineered runtime. The runtime decides what may be known, what may be sent, what may be stored, what may be acted on, and what must wait for stronger evidence or human authority.

The product objective is one coherent assistant experience backed by a durable kernel. The visible interaction can remain fast and conversational while module-local reasoning and reflection lanes decompose, schedule, check, audit, and resume deeper work. This is the practical meaning of Model Agnostic Delayed Reasoning Effort: useful local intelligence should come from architecture, context discipline, and bounded autonomy, not from blind dependence on one remote provider or one large model.

MADRE is therefore not a chatbot, not a prompt collection, not a wrapper around one vendor API, and not a task-management application. It is the runtime layer that applications use to coordinate models, internal actions, memory, knowledge, policies, and interfaces without surrendering the user's private domain to cloud-first AI infrastructure.

1.2 Why **MADRE** Agentic System?

- **Data Sovereignty:**

The first design obligation of **MADRE** is to keep the user's data under the user's control. Contemporary AI services often make private life, professional work, organizational knowledge, and creative material flow through remote systems by default. That default is not neutral. It

concentrates power in the companies that own the infrastructure, the logs, the model services, the policy switches, and the billing relation.

MADRE refuses to make surveillance a prerequisite for assistance. Local-first operation is not nostalgia and it is not a feature checkbox. It is a security boundary, a political boundary, and an engineering discipline. Private conversation, local knowledge, organizational memory, and sensitive work must remain local unless an explicit policy, classification, minimization, and audit path says otherwise.

Internet access is treated as a risk boundary, not as a casual technical detail. Prompt injection, credential exposure, external action abuse, provider lock-in, and remote data retention are not exceptional edge cases. They are normal failure modes of cloud-first agentic systems. **MADRE** 's architecture must make those failures harder by default and visible when they cannot be eliminated.

- **Free Software Duty:**

MADRE is released under GPL-3.0 because source availability alone is not enough. The project aims to provide usable agentic infrastructure for people and organizations that cannot, or should not, depend on Big Tech platforms for every intelligent interface they build.

The ethical argument is direct. The current generation of AI models was trained on an enormous inheritance of human language, labor, research, art, software, and social knowledge. Much of that inheritance was collected without meaningful consent, attribution, compensation, or democratic governance. A public-interest response cannot be limited to complaining about extraction while leaving all practical infrastructure inside the same corporate channels.

MADRE 's answer is to build an open framework that helps developers and creators run useful systems on ordinary machines, inspect how decisions are made, replace model runtimes and authorized integrations, and share improvements without asking permission from a platform owner. The goal is not to imitate proprietary AI products locally; the goal is to make a different kind of agentic infrastructure possible.

- **Model Agnosticism:**

Model agnosticism is a technical requirement and an anti-lock-in principle. No single model, host, vendor, format, or prompt convention should define **MADRE** 's architecture. Models improve, fail, disappear, become expensive, change license terms, and carry different bias and safety profiles. A serious runtime must survive that instability.

MADRE treats model invocation as a governed internal action behind an explicit runtime boundary. The runtime must be able to choose light local models for low-effort work, stronger models for demanding reasoning, verification models for checks, embedding models for retrieval, and authorized runtime bindings for different model families. The kernel's job is to manage that variety without letting any model become the authority over policy, memory, actions, or truth.

This posture also rejects AGI mythology as an engineering foundation. LLMs do not need to be treated as minds to be useful. They can be treated as language machines inside a disciplined system that supplies context, decomposes work, checks outputs, records evidence, and escalates uncertainty.

- **Resource Discipline:**

Local-first AI will fail if it assumes everyone owns specialized hardware or can pay for unlimited remote inference. **MADRE** must be useful on ordinary commercial devices, and it must degrade intelligently when resources are scarce.

The answer is not to pretend small models are magically equivalent to large ones. The answer is architecture: Complex Agent foreground lanes, reasoning and reflection lanes, task decomposition, caches, short-lived specialized agents, context minimization, repeated checks where they matter, and durable memory that avoids recomputing the same work. **MADRE** should spend compute like a scarce civic resource, not like an infinite billing stream.

This is also an ecological position. Sending every minor task across the internet to high-consumption data centers is not a harmless default. A runtime that can do more locally, reuse knowledge, and postpone expensive reasoning until it is justified is a better engineering object and a more responsible social object.

- **Knowledge Governance:**

Agentic systems become dangerous when they blur the boundary between text, knowledge, memory, instruction, evidence, and action. **MADRE** must make those boundaries explicit. A conversation turn is not a verified fact. A generated summary is not a source. An action result is not an instruction. A learned pattern is not automatically policy.

Knowledge must be isolated by domain, provenance, sensitivity, and use. The user must be able to understand what knowledge is being used, correct it, detach it, delete it, or prevent it from crossing into another context. This is especially important for organizations and collectives that need their own reliable sources to resist the default bias of models trained on dominant social, commercial, and colonial narratives.

MADRE should support learning, but only as governed evolution. Memories, derived knowledge, prompt improvements, routing rules, workflows, and model adaptation candidates must pass through quarantine, verification, promotion, and rollback boundaries. A system that learns without control is not autonomous in a useful sense; it is merely uncontrolled.

- **Distributed Reasoning:**

MADRE’s cognitive thesis is pragmatic. Human work does not happen as one monolithic response. We answer quickly, defer hard problems, retrieve memory, ask for help, test hypotheses, revise plans, and learn from outcomes. Agentic software can mirror that shape without pretending to reproduce the human mind.

The foreground lane of a Complex Agent exists to preserve conversational flow and user attention. It should clarify, triage, summarize known local facts, and create reasoning work when an answer needs deeper analysis. Reasoning and reflection lanes can then plan, search, verify, invoke bounded internal actions, compare alternatives, and return results when there is enough evidence.

The illusion of one coherent assistant should emerge from many controlled runtime actors. That illusion is useful for the user, but it must not hide the engineering truth: every agent, model invocation, action invocation, memory update, and learning proposal needs an accountable boundary. **MADRE** is designed so the assistant can feel unified without becoming opaque.

Specification Scope

MADRE is a local-first, model-agnostic runtime kernel for constructing agentic systems whose useful behavior is produced by software architecture rather than by treating language models as autonomous intelligence. The runtime separates Complex Agent foreground continuity from deeper reasoning and reflection lanes, governs context before it reaches models or internal actions, constrains executable behavior, keeps memory and knowledge inspectable, and requires learning to pass through quarantine, evidence,

promotion, and rollback boundaries.

This specification defines the research, product, and architecture baseline for MADRE. It states the system problem, expected contribution, stakeholder concerns, requirement domains, architecture views, safety model, policy model, and evaluation surface used to assess runtime behavior. The product is an open, extensible foundation for private and affordable agentic software on ordinary local devices, with optional remote services and integrations treated as governed risk boundaries rather than defaults.

The scope of MADRE comprises runtime purpose, system boundary, requirement families, architecture obligations, risk model, quality attributes, validation strategy, and product-level authority semantics.

The design identifiers D-001 through D-013 are internal traceability anchors inside this dossier. They identify product rationale, runtime obligations, safety boundaries, and evaluation surfaces at architecture level without making prior side specifications authoritative.

Canonical Terms

Term	Meaning in MADRE
Runtime kernel	The deterministic local coordination layer that registers modules, routes requests, owns global scheduling, resource limits, the Internal Action Registry, policy root, audit root, persistence coordination, and lifecycle boundaries.
Reasoning module	A governed runtime unit for a bounded reasoning domain, action family, topic, or scope. A module owns local authority, policies, queues, workflows, audit stream, knowledge/cache boundaries, assignable agents, and a system agent.
MADRE core module	The central reasoning module that receives interaction, preserves continuity, chooses fast response, clarification, delay, block, or delegation, and creates dynamic reasoning plans.
MADRE agent	A model-agnostic runtime actor inside a module. An agent executes tasks through routines and invokes models or internal actions through strategy, but it is not a model and not a model runtime binding.

Term	Meaning in MADRE
Agentic workflow	Module-level orchestration that coordinates agents and routines under module authority and kernel policy.
Agentic routine	Agent-level reusable recipe for a habitual task or action.
Reasoning plan	Dynamic plan produced from a reasoning request; it may select workflows, decompose tasks, assign agents, and define audit or recovery expectations.
Access surface	A CLI, local console, GUI, speech surface, or application interface that exposes the kernel without becoming runtime authority.
Foreground lane	The Complex Agent lane that preserves flow, clarifies intent, answers only within safe known context, and creates delayed work when stronger evidence is required.
Reasoning lane	Durable module-local work used for analysis, evidence gathering, verification, internal action invocation, or higher-effort model invocation.
Context bundle	A bounded, sourced, classified, minimized, and revocable selection of information prepared for a model invocation, internal action, or reasoning task.
Internal action	A typed runtime-defined executable behavior made available only through declared contracts and policy checks.
Candidate	An inactive proposal for promotion, persistence, behavior change, response acceptance, artifact materialization, memory, knowledge, routine, workflow, routing, prompt, policy, or model-strategy change.

Term	Meaning in MADRE
Artifact	A materialized output such as a file, document, code patch, report, image, dataset, PDF, snapshot, or final buffer/document. Ordinary conversation text is not an artifact unless materialized.

Specification Baseline

Version	Status
Beta v0.1	Research, product, and architecture specification baseline for local-first, model-agnostic delayed reasoning. The baseline covers product requirements, architecture obligations, safety boundaries, traceability, and evaluation criteria.

Research Foundation

2.1 Research Problem

Modern language-model systems often combine private data, model calls, action execution, memory, and remote infrastructure inside opaque conversational interfaces. This makes useful demonstrations easy to produce, but it weakens local ownership, traceability, autonomy control, error recovery, and the ability to replace models or providers.

MADRE addresses this as a Software Engineering problem: how to design a local-first runtime in which language models remain fallible replaceable functions, while the software system owns policy, context, memory, internal actions, audit, recovery, and learning boundaries. The research problem is not to prove that models reason, but to define an architecture that extracts useful language behavior without surrendering system authority to model output.

2.2 Research Questions

ID	Research question
RQ-1	How can a local-first runtime separate immediate interaction from delayed reasoning without losing continuity, accountability, or user control?
RQ-2	How can model-agnostic contracts prevent provider-specific assumptions from entering policy, memory, action authority, and truth management?
RQ-3	What traceability structure is sufficient to connect MADRE ’s product claims, stakeholder concerns, runtime requirements, architecture decisions, and validation evidence?
RQ-4	How can context, memory, and learned candidates remain useful while preserving provenance, minimization, revocation, correction, quarantine, and rollback boundaries?
RQ-5	What integrated behavior demonstrates that MADRE is a governed runtime rather than a conversational wrapper?

2.3 Hypotheses

ID	Hypothesis
H-1	Module-local foreground, reasoning, and reflection lanes can preserve conversational responsiveness while improving reliability for tasks that require evidence, verification, action execution, or stronger context.
H-2	Model-agnostic contracts can reduce provider lock-in when policy, memory, context, and action execution remain owned by the runtime kernel.
H-3	Context governance with provenance, classification, minimization, and revocation can reduce leakage and misuse without eliminating useful specialized reasoning.
H-4	Quarantined learning can improve system adaptation while preventing model output from silently rewriting protected behavior or trusted knowledge.
H-5	A local event and audit model can make delayed reasoning inspectable and recoverable enough for ordinary local operation.

2.4 Expected Contributions

MADRE’s expected contribution is a reference architecture for local-first, model-agnostic delayed reasoning. The contribution combines Complex Agent foreground, reasoning, and reflection lanes, governed context bundles, explicit model invocation and internal action contracts, quarantined learning, and auditable recovery paths.

The novelty is not a new language model. It is the engineering arrangement that keeps language models useful while preventing them from becoming sources of truth, authority, policy, memory, or action. The architecture is intended to be useful for personal assistants, developer systems, organizational knowledge workers, domain systems, and local interfaces that require control over private knowledge.

2.5 Novelty And Validation Strategy

The first research claim is that a useful agentic system can be validated as a software architecture before it is validated as a broad automation product. **MADRE**’s novelty is the conjunction of local-first execution, model-runtime and action boundaries, delayed reasoning, governed context, explicit policy outcomes, candidate-only learning, and audited recovery.

The validation strategy examines whether a local request can enter through bounded access surfaces, be triaged without unsupported claims, become durable delayed work when stronger evidence is needed, obtain a governed context bundle, cross a replaceable model runtime boundary, invoke a bounded internal action when policy allows it, produce an evidence-aware result candidate, expose status and audit evidence, and support correction, invalidation, supersession, rollback, or deletion/detachment.

The validation strategy treats negative paths as first-class evidence. Prompt or source injection must remain source data. Sensitive or unrelated material must be excluded from context. Model output must not become truth, policy, action authority, memory, or active knowledge by itself. Learning, prompt, routing, workflow, memory, and knowledge proposals remain inactive candidates unless a defined promotion boundary accepts them.

2.6 Related Work And Evidence Map

Evidence family	Relevance to MADRE	Specification use
Local-first software	Establishes user ownership, offline usefulness, privacy, longevity, and optional synchronization as product concerns.	Supports data sovereignty and local-first architecture claims.
Requirements engineering	Provides structure for measurable requirements, acceptance criteria, verification methods, and source traceability.	Supports the SRS and traceability model.
Architecture description	Provides stakeholder, concern, viewpoint, view, correspondence, and decision-rationale concepts.	Supports multi-view architecture description.
LLM and agent security	Identifies prompt injection, sensitive disclosure, supply-chain risk, data poisoning, excessive agency, vector weakness, misinformation, and resource consumption.	Supports threat and policy modeling.
Software quality models	Provide vocabulary for reliability, maintainability, portability, security, performance efficiency, usability, and quality-in-use evaluation.	Supports the quality attribute model.
AI evaluation research	Provides multi-metric and evidence-oriented evaluation patterns for generated outputs, retrieval quality, and agentic task performance.	Supports MADRE-native benchmark families.

Product Definition

3.1 Executive Definition

MADRE is a local-first, model-agnostic runtime kernel for governed agentic applications. It coordinates Complex Agent foreground continuity, delayed reasoning, context construction, model invocation, bounded internal action use, memory and knowledge handling, candidate learning, audit, and recovery under explicit product authority boundaries.

The product is specified as a runtime foundation rather than as a single assistant interface. A user-facing application may present a coherent conversation, but the kernel is responsible for separating immediate response from work that requires additional evidence, stronger reasoning, policy checks, or recoverable state transitions.

Language models are treated as replaceable runtime functions. They may generate candidate content, perform reasoning tasks, summarize evidence, or support classification, but they do not own policy, memory, source authority, action authority, or product truth. Those responsibilities remain architectural properties of the runtime.

3.2 Product Rationale

MADRE Agentic System

Rationale	Product implication
Local control	Private data, local knowledge, and sensitive work remain under local authority unless a policy-governed transfer is explicitly permitted.
Model substitutability	No model provider, host, prompt convention, or output format defines runtime truth, policy, memory, or action authority.
Delayed reasoning	Work that requires evidence, verification, resource control, or stronger context is represented as durable module-local reasoning work rather than as an unsupported foreground answer.
Context governance	Context is selected, minimized, classified, sourced, scoped, provenanced, and revocable before it is provided to a model invocation, internal action, or reasoning task.
Bounded action use	Internal actions, external actions, memory mutation, and workflow execution are allowed only through declared authority boundaries, policy outcomes, audit, and recovery expectations.
Candidate learning	Memory, knowledge, prompt, routing, workflow, and learning changes begin as inactive candidates until a defined promotion rule accepts them.

3.3 Product Design

3.3.1 Product Scope

MADRE provides a disciplined coordination layer underneath agentic applications. The kernel routes conversation, records delayed work, constructs bounded context, selects model invocation paths, controls bounded internal actions, manages candidate memory and knowledge, records audit evidence, and exposes recovery semantics.

The primary audience is developers and organizations building private, domain-aware, extensible assistant systems. Applications built on **MADRE** may vary in interface, domain, and policy, but they inherit the same authority model: the runtime kernel owns context, policy, action invocation, audit, and recovery boundaries.

3.3.2 What MADRE Is

- A local-first runtime kernel for agentic systems.
- A model-agnostic coordination layer for multiple model families.
- A delayed-reasoning architecture that keeps conversation responsive while deeper work happens through module-local reasoning lanes.
- A policy-aware boundary for context, internal actions, memory, learning, and remote transfer.
- A developer foundation for building assistants, domain systems, and workflow-capable applications on top of governed local intelligence.

3.3.3 What MADRE Is Not

- **MADRE** is not a chatbot product. Chat is one possible surface over the kernel, not the kernel's definition.
- **MADRE** is not a prompt wrapper. Prompts are runtime inputs, not an architecture.
- **MADRE** is not a vendor abstraction layer whose purpose is merely to swap API providers.
- **MADRE** is not an unrestricted automation agent. Internal actions, external actions, memory mutation, and learning require explicit boundaries.
- **MADRE** is not a passive record of interaction history. The product is the governed runtime kernel and its architectural obligations.

3.3.4 Core User Value

MADRE lets an application answer quickly without pretending every answer can be safe, complete, or well-reasoned in the foreground. The Complex Agent foreground lane keeps the conversation alive: it can clarify intent, state known local facts, name uncertainty, and trigger deeper work. Module-local reasoning lanes perform the work that requires decomposition, evidence, internal actions, verification, or stronger context.

This separation gives users a better contract. They are not forced to choose between a quick hallucination and a slow black box. They get immediate conversation when that is appropriate, delayed reasoning when that is necessary, and inspectable boundaries when the system touches sensitive data or meaningful actions.

3.3.5 Design Principles

- **Local-first, not local-only:** Local execution is the default because privacy, cost, resilience, and agency require a strong local base. Remote services may exist only as explicitly governed integrations with classification, minimization, policy, provenance, and audit.
- **Model-agnostic by contract:** Models are replaceable runtime functions. The kernel must not let any model provider define the system's truth model, policy boundary, memory format, or action authority.
- **Foreground lane, durable reasoning:** The Complex Agent foreground lane should preserve conversational flow. Expensive, uncertain, risky, or evidence-heavy work should become scheduled reasoning that can be resumed, inspected, verified, and delivered asynchronously.
- **Context is a governed asset:** Context is not just text stuffed into a prompt. It is selected, minimized, classified, sourced, scoped, and revoked according to the purpose of the task and the authority of the actor using it.
- **Autonomy is bounded:** **MADRE** must support unattended delayed work, but autonomy is legitimate only when the action is allowed, logged, recoverable for its risk level, and blocked from crossing stronger boundaries without authorization.
- **Learning starts quarantined:** Useful systems should improve from use, but learned candidates cannot quietly rewrite protected behavior. Memories, knowledge, prompts, routing preferences, and workflow improvements must be reviewable, promotable, reversible, and separable.
- **Interfaces do not own the architecture:** The *madre* CLI, local read-first inspection console, graphical UI, speech surfaces, or domain applications are access surfaces. They must expose and operate the kernel without becoming the source of runtime authority.

3.3.6 Product Boundaries

The product boundary covers runtime responsibilities and architecture obligations: local-first operation, model-agnostic invocation, Complex Agent lane coordination, governed context, bounded internal actions, memory and knowledge isolation, candidate learning, audit, and recovery.

MADRE may expose many application forms: personal assistants, research systems, developer environments, organizational knowledge workers, or domain-specific automation environments. Those applications may differ in interface and policy, but they must inherit the same kernel commitments: local-first operation, replaceable models, governed context, bounded internal actions, inspectable memory, quarantined learning, audit, and recovery.

Operational Scenarios

4.1 Foreground Lane Continuity

The foreground lane maintains the conversational thread, assigns or updates the current topic, asks for clarification when intent or risk is unclear, and answers only when the needed information is already available within the active context boundary. When a task needs research, verification, internal actions, broader context, or higher-effort reasoning, the foreground lane records the need as delayed work instead of forcing a premature answer.

4.2 Delayed Research

Research work begins when a Complex Agent identifies a question that cannot be answered safely from current local knowledge. The runtime checks whether sufficient local information exists, constructs a bounded reasoning request, and uses module-local reasoning execution to gather, compare, and verify evidence. Verified research results may become source material for governed knowledge extraction.

4.3 Knowledge Extraction

Knowledge extraction converts files, folders, reports, or other source material into auditable knowledge candidates. Extracted material remains separable from general memory until provenance, scope, sensitivity, usefulness, and correction rules are established. The user must be able to detach, correct, or delete a knowledge area without collapsing unrelated domains.

4.4 Specialized Conversation Mutation

When a conversation requires domain-specific expertise, the foreground lane may request a specialized conversational mode with a scoped context bundle and appropriate model configuration. The specialized session must remain named, trackable, and constrained by the same context and policy rules as delayed reasoning work.

4.5 Workflow Creation

MADRE is intended to support reusable workflows created through a user-facing contract rather than through hidden prompt chains. Workflow definitions may combine conditional logic, loops, prompts, internal action invocations, explicitly authorized external actions, model invocations, and other workflows, but each executable step remains a declared action subject to policy, audit, and recovery expectations.

4.6 Developer Extension

Developers should be able to add models, internal actions, authorized external actions, projections, context policies, workflows, interfaces, and non-deterministic analysis helpers without rewriting the kernel.

Extension points must preserve **MADRE** 's model-agnostic design: runtime authority remains in explicit contracts and policy boundaries, not in a specific provider, framework, or model family.

Conceptual Architecture

5.1 Formal Requirements

This chapter states architectural requirements at product level.

Requirement	Architectural meaning
Local-first operation	The kernel must be able to run useful workflows on local user-owned infrastructure before optional remote integrations are considered.
Model agnosticism	All model execution enters through replaceable model invocation contracts, never through a vendor-specific core design.
Delayed reasoning	The runtime must separate immediate conversation from deeper work that needs decomposition, evidence, verification, internal actions, or more compute.
Context governance	Context must be selected, minimized, classified, provenanced, scoped, and revocable before it reaches models or internal actions.
Bounded internal actions	Executable behavior must be declared, policy-checked, audited, and constrained by side effect and recovery rules. Typed runtime-defined internal actions are the canonical low-risk action form.
Memory/knowledge isolation	Personal memory, domain knowledge, untrusted source material, and learned candidates must not collapse into one undifferentiated prompt store.
Learning quarantine	Runtime learning begins as candidate material and changes active behavior or protected knowledge only through a defined promotion boundary.
Inspectable execution	The CLI, local developer console, logs, audits, and read models must make runtime behavior understandable without bypassing policy.

5.2 High-Level Architecture

MADRE is organized around a deterministic runtime kernel that coordinates governed reasoning modules through responsibility boundaries rather than through a fixed topology.

The central architectural rule is simple: the model is never the system. Models are invoked by the runtime under context, policy, internal action, memory, and audit constraints. Agents are runtime actors that combine a purpose, context boundary, model-selection strategy, action permissions, memory scope,

and lifecycle. They may be short-lived, persistent, specialized, conservative, speculative, or workflow-bound, but they do not own the kernel. Model runtime bindings are infrastructure, not agents.

The kernel coordinates modules; it does not become the reasoning actor for every task. System-level reasoning belongs in governed modules. The MADRE core module handles incoming interaction, continuity, foreground-lane response, clarification, dynamic planning, delegation, and delayed-work decisions under kernel policy. Other reasoning modules handle bounded domains such as local knowledge, research, code writing, image generation, or resource optimization.

5.2.1 Conceptual Components

Component	Responsibility
Access surfaces	Present the system through CLI, local read-first inspection console, graphical UI, speech, or domain applications without becoming the source of runtime authority.
MADRE core module	Maintains conversational continuity, clarifies intent, handles known low-risk answers through its foreground lane, creates reasoning plans, delegates to modules, and decides when deeper work is needed.
Runtime kernel	Coordinates module registration, routing, scheduling, resource limits, policies, action registries, durable records, recovery, and lifecycle.
Reasoning modules	Execute bounded reasoning domains through module-local agents, workflows, routines, queues, policies, audit streams, and knowledge/cache boundaries.
Context and knowledge	Provides scoped context bundles, memory, domain knowledge, provenance, classification, redaction, and knowledge correction paths.
Local model runtime boundary	Selects local or explicitly authorized model execution through model profiles, model invocation actions, decoding configuration, session/cache policy, runtime bindings, and inference evidence.
Internal Action Registry	Exposes typed, bounded, auditable internal actions with declared inputs, outputs, side-effect class, policy requirements, audit evidence, and recovery expectations. External actions are exceptional risk boundaries, not the default execution path.
Learning and evolution	Captures improvement candidates, evaluates them, and promotes only policy-authorized changes into active memory, knowledge, routing, or workflow behavior.
Observability and recovery	Gives developers and authorized users a read-first view of tasks, decisions, audit, metrics, failures, and rollback options.

5.2.2 Responsibility Layer Diagram

Figure 5.1 presents the primary responsibility structure of **MADRE**. The diagram is not a sequential request pipeline. It describes the ownership boundaries that constrain all runtime behavior.

The **MADRE** runtime kernel is the deterministic system coordinator. It owns access-surface entry, module registration, module routing, scheduling, resource assignment, global policy, audit root, durable runtime state, and lifecycle boundaries. The kernel does not become the reasoning actor for ordinary tasks. It selects, activates, limits, and observes reasoning modules.

Reasoning modules are the runtime units that perform domain-bounded reasoning. The Core Module is one reasoning module among others, although it is the default first receiver for ordinary user interaction because it specializes in conversational continuity, clarification, general planning, and delegation. Other modules may specialize in research, knowledge extraction, code, media, resource optimization, or domain-specific work. A module may answer quickly, create delayed work, invoke internal actions, or reflect on its own outcomes when its local policy and assigned resources allow it.

Executable behavior is not exposed as arbitrary external tools. The normal execution path uses an Internal Action Registry composed of typed runtime-defined functions with declared inputs, outputs, side-effect class, policy requirements, audit evidence, and recovery expectations. The first reference implementation may implement these actions on a JVM runtime, but JVM binding is not an architecture-level authority. This registry is the canonical low-risk action mechanism. External services, host commands, provider-backed tools, remote model services, or arbitrary plugin surfaces are not part of the default path and require stronger authority boundaries.

The Local Model Runtime Boundary is below the module and action layer. It represents model profiles, decoding configuration, session/cache policy, device/resource policy, runtime bindings, and inference evidence. It is not a chatbot server and not an architectural authority. Generated model content returns to the module as candidate material subject to verification, audit, and promotion rules.

5.2.3 Concurrent Agent Lanes

MADRE separates immediate interaction from deeper reasoning, but this separation is not modeled as two global subsystems. It is modeled as a recurrent execution pattern inside Complex Agents.

A Complex Agent may maintain three concurrent lanes. The foreground lane preserves conversational continuity, answers only when safe known context is sufficient, asks for clarification when intent or risk is unclear, and creates deferred work when more evidence is required. The reasoning lane performs planning, decomposition, evidence gathering, action invocation, verification, and result construction under module authority. The reflection lane reviews recent work, observes delayed outcomes, detects failure patterns, and proposes inactive learning candidates.

This pattern may exist in any reasoning module. The Core Module uses it for general interaction and delegation. A research module may use it for evidence comparison and verification. A knowledge module may use it for extraction, provenance checks, correction, and deletion semantics. The kernel schedules and constrains these lanes through module routing, resource limits, durable state, policy, and audit, but the reasoning behavior itself belongs to module-local agents.

5.3 Runtime Data Flow

Runtime data flow in **MADRE** is concurrent, governed, and module-centered. A local request enters through an access surface and is submitted to the kernel. The kernel records the request, applies the relevant entry policy, and routes it to the Core Module by default or to another registered module when routing evidence supports that decision.

Within the selected module, a Complex Agent may produce an immediate foreground response, create or continue delayed reasoning work, invoke internal actions, request local model inference, and reflect on recent outcomes. These activities are not independent architectural layers. They are coordinated lanes inside module-local agents and constrained by module policy, kernel policy, resource assignment, durable state, audit, and recovery rules.

The following subsections describe the canonical request narrative, the governed model/action sequence, and the data-authority rules that prevent user input, source material, model output, action results, or learning candidates from promoting themselves into trusted state.

5.3.1 Canonical Request Narrative

The canonical user request begins when an access surface submits a local request to the kernel. The kernel does not interpret the request as trusted knowledge or policy. It records the event, applies entry constraints, and routes the work to the Core Module unless another module is already selected by session state, policy, or module registry evidence.

The receiving module evaluates the request through its module-local Complex Agent. If the foreground lane has enough safe local context, it may produce an immediate answer. If intent, scope, or risk is unclear, it asks for clarification. If the request requires evidence, stronger reasoning, internal actions, model inference, or broader context, the reasoning lane creates or continues durable work. Reflection may run after the response or after delayed outcomes to propose inactive learning, routing, prompt, workflow, memory, or knowledge candidates.

Every significant step emits audit evidence. Context bundles, action invocations, model invocations, candidate results, policy outcomes, failures, correction, rollback, invalidation, supersession, and deletion/detachment remain inspectable through read-first surfaces.

5.3.2 Governed Model And Action Sequence

When module work requires executable behavior, the module invokes an internal typed action. The default action mechanism is local, typed, runtime-defined, policy-gated, and auditable. It is not an open plugin socket and not a network-exposed external action protocol.

A model invocation is one specialized internal action. Before invocation, the module supplies a governed context bundle, a model strategy, and the applicable action policy. The input compiler prepares the model-specific representation without allowing the model provider, template, or runtime format to define **MADRE**'s authority model. The decode controller applies the selected inference strategy, records sampling and cache configuration, and returns generated material as a candidate.

Action results and model outputs are observations or candidates. They may inform the next reasoning step, but they do not authorize actions, mutate memory, update trusted knowledge, change routing, modify workflows, or promote learning. Promotion requires the appropriate authority boundary, verification path, audit record, and recovery expectation.

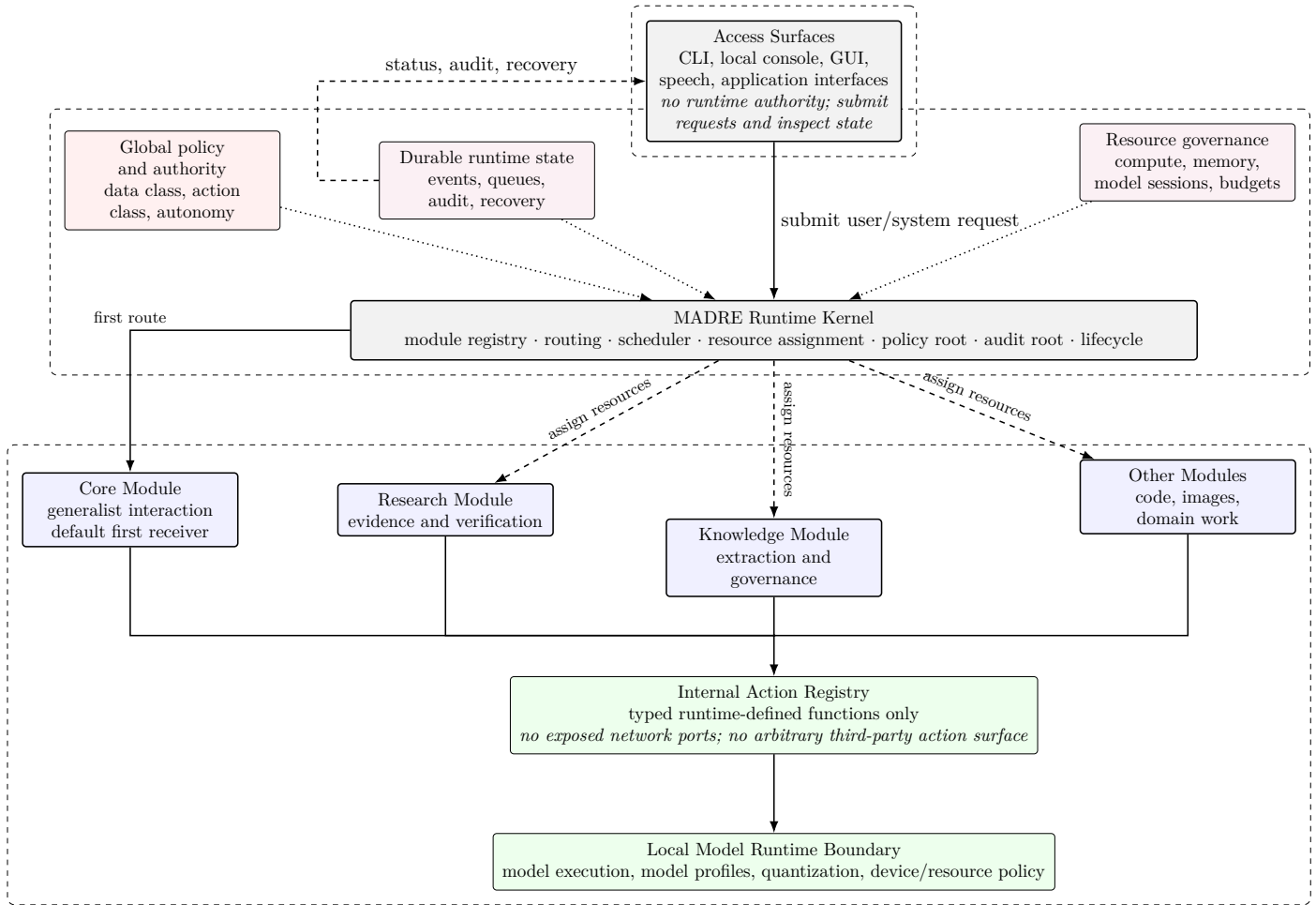


Figure 5.1: MADRE responsibility layers: kernel coordination, reasoning modules, internal actions, and local model runtime.

Figure 5.1 defines the top-level responsibility model. Access surfaces are intentionally outside runtime authority: they submit requests and inspect state, but they do not own policy, memory, routing, scheduling, or action-authority decisions. The kernel is the only system-level coordinator. It registers modules, routes requests, assigns resources, owns global policy and audit roots, owns action authority, and maintains durable lifecycle state.

The module layer is the reasoning layer. The Core Module is shown first because ordinary user prompts are routed there by default, not because it is outside the module system. Research, knowledge, code, media, or domain modules are peers selected when the registry, policies, and current reasoning plan indicate a better fit. Internal actions and local model execution appear below the module layer because they are invoked by authorized module behavior, not by access surfaces or model output directly.

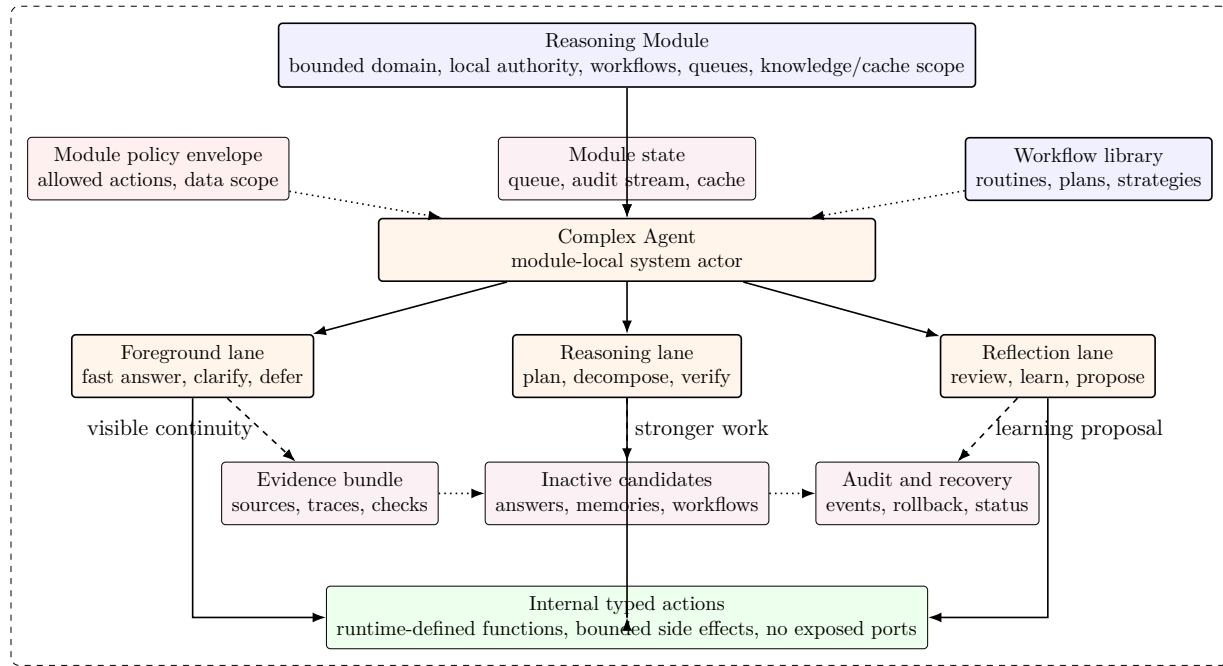


Figure 5.2: Reasoning module internal structure with a Complex Agent using concurrent foreground, reasoning, and reflection lanes.

Figure 5.2 expands the internal structure of a reasoning module. A module owns a bounded domain, local policy envelope, queue, audit stream, cache, workflow library, and at least one module-local system agent. The Complex Agent pattern is used to avoid the false assumption that useful intelligence is produced by one prompt and one response.

The foreground lane, reasoning lane, and reflection lane may operate concurrently. The foreground lane protects user continuity. The reasoning lane protects evidence, decomposition, verification, and stronger work. The reflection lane protects adaptation by turning observations into inactive candidates rather than silently rewriting runtime behavior. These lanes do not increase agent authority. They remain constrained by module policy, kernel policy, assigned resources, audit requirements, and promotion boundaries.

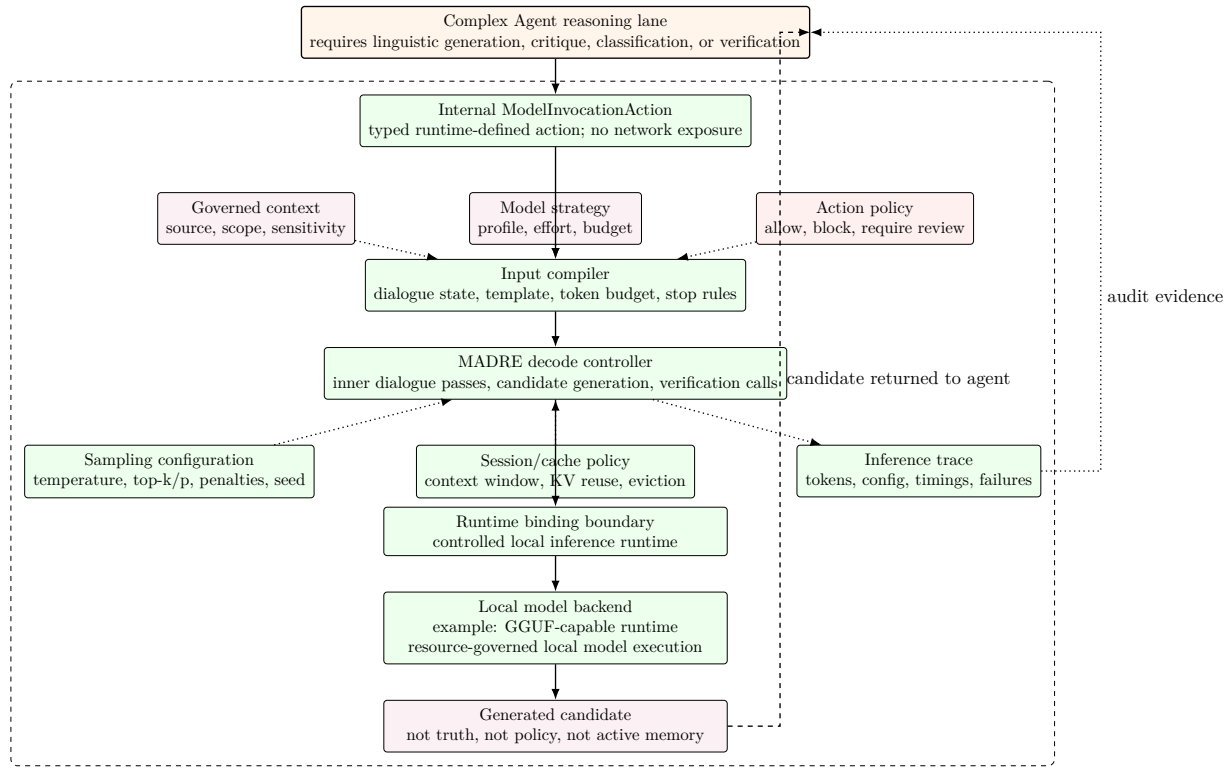


Figure 5.3: Internal model invocation action with MADRE-owned decode control and no exposed server boundary.

Figure 5.3 describes model invocation as an internal action rather than as an external server call or chat-completion product boundary. A Complex Agent requests linguistic generation, critique, classification, or verification through a typed `ModelInvocationAction`. That action receives governed context, a model strategy, and an action policy decision before input compilation.

The input compiler transforms dialogue state, context, templates, token budgets, and stop rules into the representation required by the selected local model runtime. The **MADRE** decode controller may perform inner-dialogue passes, candidate generation, verification calls, and budget-aware generation without surrendering control to a provider-specific chat abstraction. Sampling configuration, session/cache policy, and inference trace are explicit runtime evidence. The runtime binding boundary may use any later-approved local inference mechanism, but the selected backend remains an implementation detail below the model runtime boundary.

The generated output is a candidate. It is not truth, policy, active memory, or executable authority. It returns to the agent and module for verification, audit, and possible promotion according to the same data and authority rules that govern the rest of **MADRE**.

5.3.3 Data And Authority Rules

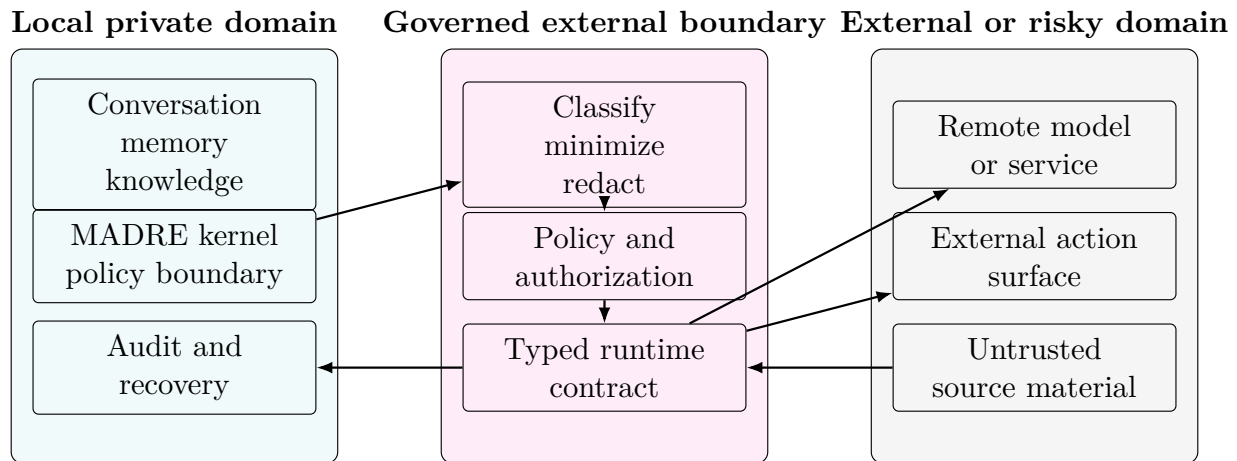
- User input enters as conversation, not as policy, memory, source truth, action authority, or model authority.
- Retrieved or extracted material enters with provenance, domain scope, sensitivity, confidence, and correction metadata.

- Context bundles are constructed, minimized, classified, scoped, and revocable before they reach an agent lane, internal action, or model invocation.
- Model output enters as generated candidate material that requires the appropriate level of verification before it can become an answer, record, action input, memory, knowledge, or learning candidate.
- Internal action results enter as bounded observations. They become trusted evidence only according to the action contract, verification path, and audit record.
- External action surfaces, host commands, remote services, arbitrary filesystem mutation, credentials, and irreversible effects require stronger authority boundaries than ordinary internal actions.
- Learning output enters quarantine first and cannot directly rewrite active runtime behavior.

5.3.4 Trust Boundary Diagram

The trust boundary diagram distinguishes the ordinary local governed domain from optional external or risky domains. The default execution path remains inside the local governed domain: access surfaces, kernel coordination, reasoning modules, internal typed actions, local model runtime, audit, and recovery.

The external boundary exists for exceptional external actions: remote services, provider-backed models, host-command execution, arbitrary filesystem mutation, network access, untrusted source acquisition, or third-party integrations. These are not ordinary internal actions. They require classification, minimization, explicit policy authorization, audit, and a recovery or blocking expectation appropriate to their risk class.



5.3.5 Access Surfaces

MADRE exposes access surfaces without allowing those surfaces to define the kernel. The **madre** CLI submits local requests and exposes status or result evidence. A local read-first inspection console may expose sessions, queued work, context bundles, policy outcomes, model evidence, internal action evidence, candidates, audit, recovery, and runtime state. These surfaces operate under the same policy and authority model as every other runtime interaction.

The inspection console is not a privileged mutation path. Its transport is intentionally not fixed at architecture level. If a later implementation uses HTTP, sockets, embedded UI, or another mechanism,

that choice must be justified by deployment and security decisions and must preserve local-first, read-first, policy-governed behavior.

Graphical, speech, product-specific, or integration surfaces remain access surfaces over the runtime rather than privileged paths around it.

Safety And Evolution Policy

6.1 Runtime Safety

MADRE is intended to run useful delayed work while the user is not constantly present. That requirement makes safety an architectural property, not a documentation afterthought. The system must be able to explain what it did, why it did it, what data it used, what boundary allowed it, and how the result can be corrected or rolled back when the action is reversible.

6.1.1 Policy Before Action

The kernel must not expose executable behavior merely because a model can request it. Internal actions, external calls, memory updates, knowledge promotion, workflow execution, and learning changes need explicit policy before execution. The safe path should be the normal path, and safe autonomous improvement should not require constant user interruption. Unsafe paths should be technically isolated, auditable, recoverable where possible, and visible to the responsible developer or operator.

Typed runtime-defined internal actions represent the canonical bounded action form. They may be allowed only through a policy-gated action boundary. Remote action, host commands, arbitrary filesystem access, mutation actions, provider-backed integrations, active learning promotion, and workflow mutation require stronger authority boundaries than bounded internal action execution.

6.1.2 Autonomy Levels

Level	Meaning
Observation	Read local runtime state, existing knowledge, and safe metadata without mutation.
Draft creation	Produce summaries, candidate records, or proposed changes without promoting them into active state.
Bounded mutation	Modify authorized local state only when the action is logged, scoped, reversible where possible, and policy-permitted.
External action	Contact remote systems, use external services, or disclose data only after classification, minimization, authorization, provenance, and audit.
Critical action	Credentials, identity, legal, financial, destructive, irreversible, or executable-code actions require a stronger human authorization path.

6.2 Evolution Control

6.2.1 Evolution Rules

- Runtime learning is represented as a governed subsystem, not only as manual feedback collection.

- Learning strategies must declare what they are allowed to observe, propose, mutate, verify, and promote.
- Runtime learning cannot silently modify protected behavior.
- Learned candidates begin with source, evidence, scope, and rollback information before they are promoted.
- Knowledge promotion requires provenance, correction paths, and recovery options.
- Workflow and routing improvements may become default behavior through developer-defined checks, evaluation gates, and rollback policies, without requiring user authorization for every routine improvement.
- Model adaptation, fine-tuning, or weight updates are advanced model-evolution actions governed by stronger model-adaptation authority.
- Critical learning failures must be recoverable through snapshots, versioned state, VCS-backed records, or an equivalent rollback mechanism.

This policy keeps **MADRE** capable of evolving through conscious, checked learning without reducing improvement to constant user authorization. The final user should be interrupted as little as possible. Routine learning should be governed by developer-defined strategies, evidence, tests, promotion gates, audit trails, and rollback mechanisms; user authorization is reserved for boundaries where risk, privacy, identity, external action, or policy explicitly requires it.

D-011 fixes the quarantine rule for memory, knowledge, prompt, routing, workflow, and learning changes. Candidate creation is useful system evidence, but it is not active promotion and it cannot silently rewrite protected behavior.

System Engineering Baseline

7.1 Engineering Problem

MADRE addresses the engineering problem of coordinating language-model model invocation without allowing the model to become the system. The runtime must support foreground continuity, delayed reasoning, governed context, bounded internal actions, memory, learning, audit, and recovery while preserving local ownership of data and operational control.

The core difficulty is not only model accuracy. A useful local assistant must decide when an answer is safe in the foreground lane, when work must be delayed, which context may be exposed, which internal action may be invoked, which result requires verification, and how a mistaken or unsafe outcome can be inspected, corrected, or rolled back.

7.2 Stakeholders And Concerns

MADRE serves users who need private assistance, developers who need an extensible runtime, organizations that need governed local knowledge, and maintainers who must keep the system auditable and evolvable. Each stakeholder depends on the same architectural guarantees: model independence, local-first operation, explicit authority boundaries, understandable execution, and recoverable state.

The principal concerns are privacy, cost, autonomy control, model substitution, context minimization, data provenance, action safety, knowledge correction, learning control, operational continuity, and evidence for system behavior. A runtime that cannot expose those concerns as inspectable state cannot satisfy **MADRE**'s product objective, even if its conversational output appears fluent.

7.3 Requirement Domains

MADRE's functional baseline is organized around runtime domains. The foreground lane domain preserves conversational flow, asks for clarification, and creates deeper work when the answer requires evidence or internal actions. The delayed reasoning domain records work durably, schedules it, verifies results, and returns outcomes without erasing the foreground interaction.

The context domain selects, minimizes, classifies, scopes, and revokes information before it reaches models or internal actions. The model runtime domain invokes local or policy-authorized remote models behind replaceable contracts. The internal action domain exposes declared side effects under policy and audit. The knowledge and memory domain separates private memory, domain knowledge, untrusted source material, and learned candidates.

The learning domain keeps improvement proposals quarantined until evidence, evaluation, promotion, and rollback rules allow activation. The observability domain exposes sessions, tasks, context bundles, model invocations, action invocations, authorizations, failures, metrics, and recovery paths. The deployment domain keeps ordinary local operation viable before optional remote integrations are considered.

7.4 Traceability Surface

Each significant product claim in **MADRE** must be visible as system behavior. A claim such as local-first operation corresponds to local execution paths, storage boundaries, remote-transfer controls, and inspection evidence. A claim such as model agnosticism corresponds to replaceable model profiles, runtime contracts, and tests that prevent provider-specific assumptions from entering the kernel.

The same trace applies to safety claims. Policy before action must appear as an enforceable boundary around internal actions, external calls, memory mutation, workflow execution, and learning promotion. Governed context must appear as provenance, classification, minimization, scope, revocation, and audit. Delayed reasoning must appear as durable work records, status transitions, verification, result delivery, and recovery.

7.5 Quality Attributes

MADRE prioritizes data sovereignty, privacy, resource discipline, responsiveness, reliability, recoverability, maintainability, portability, auditability, and resistance to hallucination-driven action. These attributes are not independent decorations; they define whether the runtime fulfills its purpose.

Responsiveness is measured by preserving useful foreground conversation while heavier work is delayed. Reliability is measured by durable records, repeatable status, bounded retries, and visible failures. Maintainability is measured by the ability to replace models, internal actions, workflows, and interfaces without rewriting the kernel. Auditability is measured by the ability to explain what data, action, policy, and verification path produced an outcome.

7.6 Risk And Policy Boundaries

MADRE treats prompt injection, data leakage, excessive agency, unsafe action execution, poisoned knowledge, provider dependency, remote retention, unbounded consumption, credential exposure, and rollback gaps as architectural risks. The runtime must assume that model output can be persuasive, wrong, incomplete, unsafe, or maliciously influenced by context.

Risk control is expressed through policy boundaries. Data classification governs what may be retrieved, stored, disclosed, or sent to a model. Autonomy level governs whether a result may remain a draft, update local state, invoke an internal action, contact an external service, or require stronger authorization. Recovery expectation governs whether the action must be reversible, snapshot-backed, compensatable, or blocked.

Requirements Baseline

8.1 Requirement Taxonomy

Prefix	Requirement family
FR	Functional behavior of foreground-lane conversation, delayed reasoning, context construction, model invocation, internal actions, memory, learning, audit, and recovery.
NFR	Quality attributes such as responsiveness, reliability, recoverability, maintainability, portability, resource discipline, and auditability.
SEC	Security and trust controls, including prompt injection resistance, credential protection, action authority, and remote-transfer boundaries.
DATA	Data, knowledge, provenance, retention, deletion, correction, revocation, and quarantine requirements.
AI	Model invocation, model substitution, generated-output handling, and verification requirements.
UX	User-visible states, clarification, authorization, progress, failure, correction, and result-delivery requirements.
OPS	Local deployment, storage, backup, diagnostics, migration, and operational recovery requirements.

8.2 Functional Requirements

ID	Requirement	Acceptance criteria	Verification
FR-001	The runtime shall preserve foreground-lane conversation while delaying evidence-heavy, risky, action-dependent, or broad-context work.	Immediate answers stay within safe known context; stronger work is recorded as delayed work.	Triage scenarios; D-001, D-004.
FR-002	The runtime shall represent delayed work as durable local records with inspectable status and recovery path.	Work exposes queued, running, blocked, failed, completed, and recovered states with audit evidence.	Work-lifecycle and recovery scenarios; D-001, D-003, D-008.
FR-003	The runtime shall build context bundles from sourced, classified, minimized, scoped, provenanced, and revocable material.	Context is validated or rejected before model invocation or internal action use.	Context-governance scenarios; D-001, D-005, D-007.
FR-004	The runtime shall expose model execution through replaceable model invocation contracts.	Substitutable model profiles satisfy the same boundary behavior; failures remain inspectable.	Model substitution and failure scenarios; D-001, D-006.
FR-005	The runtime shall expose executable behavior only as bounded internal actions or explicitly authorized external actions.	Internal actions remain policy-gated, while disallowed action requests are blocked or authorization-gated.	Action policy scenarios; D-007, D-009.
FR-006	The runtime shall keep learning candidates inactive unless evidence, evaluation, promotion, and rollback rules activate them.	Proposed memory, knowledge, prompt, routing, workflow, or learning changes are inspectable but inactive.	Candidate quarantine scenarios; D-011.
FR-007	The runtime shall expose local request entry and read-first inspection.	Local CLI request/status interaction and local console inspection expose system state without bypassing runtime authority.	Local access and inspection scenarios; D-010.
FR-008	The runtime shall represent recovery outcomes explicitly.	Correction, rollback, deletion/detachment, invalidation, and supersession are distinguishable and auditable.	Recovery readout scenarios; D-003, D-008.

8.3 Cross-Cutting Requirements

ID	Requirement	Rationale	Verification
NFR-001	Local useful operation shall remain possible without mandatory remote service dependency.	Data sovereignty, cost control, and resilience.	Local execution evidence; D-001, D-002, D-006.
NFR-002	Runtime state changes shall be inspectable through local read models.	Auditability and recoverability.	Inspection review; D-003, D-010.
NFR-003	Delayed work shall be bounded by status, resource, and failure discipline.	Prevents unbounded cost, compute, storage, and queue growth.	Resource/status scenarios; D-003, D-007, D-010.
SEC-001	Prompt or source material shall not become instruction, policy, or authority without explicit classification and boundary checks.	Prompt injection and data poisoning risk.	Adversarial context scenario; D-004, D-005, D-007.
SEC-002	Policy decisions shall be explicit and auditable before action execution or promotion.	Prevents hidden allow/deny behavior and excessive agency.	Policy matrix review; D-007, D-009, D-011.
SEC-003	External, destructive, credential, legal, financial, executable, or irreversible effects shall be blocked or separately authorized.	Preserves recoverability and explicit authority over critical action.	Critical-action negative paths; D-007, D-008, D-009.
DATA-001	Knowledge areas shall support provenance, correction, detachment, and deletion semantics.	Knowledge governance and user control.	Correction and revocation scenarios; D-005, D-008, D-011.
DATA-002	Active state and accountable history shall remain distinguishable.	Allows deletion/detachment without audit erasure.	Recovery inspection scenarios; D-003, D-008.

ID	Requirement	Rationale	Verification
AI-001	Generated text shall be treated as generated output until validated by a verification or promotion path appropriate to the risk.	Hallucination and authority risk.	Verification-failure scenarios; D-001, D-006.
AI-002	Models shall not authorize actions, memory, policy, routing, or trusted knowledge changes.	Keeps runtime authority outside model output.	Model-overreach scenarios; D-006, D-007, D-011.
UX-001	Users shall be able to distinguish immediate answers, queued work, running work, blocked work, failed work, completed work, and authorization requests.	Continuity and user control.	Interaction-state review; D-004, D-010, D-011.
OPS-001	Local access surfaces shall submit requests and inspect status/audit without becoming runtime authority.	Supports operability while preserving governance.	Local access and inspection scenario; D-010.

Traceability Model

9.1 Traceability Method

Traceability in **MADRE** links the product claim, stakeholder concern, requirement, architecture obligation, validation method, and evidence status. The purpose is to keep evaluation anchored to the system thesis: local-first operation, model agnosticism, delayed reasoning, governed context, bounded internal actions, quarantined learning, audit, and recovery.

9.2 Architecture Traceability Matrix

Claim		Concern	Requirement	Decision trace			Evidence family
Local-first operation		Data sovereignty, cost, resilience	NFR-001	D-001, D-002, D-010			Local request, local status, local audit, no mandatory remote service.
Model agnosticism		Provider independence	FR-004, AI-001	D-006			Replaceable model-profile substitution and failure scenario.
Delayed reasoning		Responsiveness and quality	FR-001, FR-002	D-001, D-003, D-004			Foreground triage plus durable delayed-work lifecycle.
Governed context		Privacy and correctness	FR-003, DATA-001	D-005, D-007			Context acceptance, exclusion, source-as-data, and revocation scenario.
Bounded internal actions		Safety and recoverability	FR-005, SEC-001	D-007, D-009			Policy-gated internal action and blocked unsafe external action attempts.

Claim	Concern	Requirement	Decision trace	Evidence family
Learning quarantine	Controlled evolution	FR-006	D-011	Inactive candidate capture and inspection without active promotion.
Audit and recovery	Accountability and correction	FR-008, NFR-002, DATA-002	D-003, D-008, D-010	Status inspection, correction, invalidation, supersession, rollback, and deletion/detachment trace.

9.3 Integrated Architecture Trace

Step	Required behavior	Decision trace	Negative evidence
Local entry	Request enters through local access surface.	D-001, D-010	Internal-only execution is not enough to prove inspection.
Foreground triage	Foreground answers only safe known context, clarifies, blocks, or creates delayed work.	D-004, D-007	Evidence-heavy request is not answered as if already known.
Durable work	Delayed work is recorded with status and audit evidence.	D-003	Work is not lost or silently overwritten.
Governed context	Context is sourced, classified, minimized, scoped, provenanced, revocable, and auditable.	D-005, D-007	Sensitive, unrelated, or revoked material is excluded or blocked.
Model boundary	Model output is a candidate produced through a replaceable model invocation action.	D-006	Malformed, unsupported, or evidence-mismatched output does not become success.
Bounded internal action	Typed runtime-defined internal actions are policy-gated and audited.	D-009	Remote, host-command, filesystem, or mutation actions require stronger authority boundaries.

Step	Required behavior	Decision trace	Negative evidence
Candidate quarantine	Proposed memory, knowledge, prompt, routing, workflow, or learning material stays inactive.	D-011	Candidates cannot promote themselves or alter active behavior.
Inspection and recovery	Status, audit, failure, correction, rollback, invalidation, super-session, and deletion/detachment are visible locally.	D-008, D-010	Active state changes do not erase accountable history.

Design Decision Traceability

Design decisions D-001 through D-011 provide traceability from product requirements to architecture obligations and validation evidence. They identify accepted product architecture decisions for governed delayed reasoning, durable work, context, model invocation, policy, recovery, bounded internal actions, local access, and candidate quarantine.

Decision	Product-level effect	Architecture boundary
D-001	Defines source-backed delayed-answer behavior as a core expression of governed runtime value.	Conversation remains connected to durable work and evidence.
D-002	Establishes behavior-first architectural validation as part of the product's engineering discipline.	Observable behavior anchors architecture claims.
D-003	Requires durable delayed-work history and projected inspection state.	Work lifecycle and audit are product responsibilities.
D-004	Requires conservative foreground triage.	Unsupported answers become clarification, policy block, or delayed work.
D-005	Requires governed context acceptance and rejection.	Context is sourced, classified, minimized, scoped, provenanced, and revocable.
D-006	Requires replaceable model invocation evidence.	Models remain runtime functions, not authorities.
D-007	Requires auditable policy outcomes before action execution.	Action execution follows policy rather than model instruction.
D-008	Requires recovery as an audited semantic transition.	Correction, rollback, invalidation, supersession, deletion, and detachment have distinguishable meaning.
D-009	Defines bounded internal actions as the low-risk executable reference.	Action execution is policy-gated, auditable, and recoverability-aware.

Decision	Product-level effect	Architecture boundary
D-010	Defines CLI request/status interaction and local read-first console inspection as canonical local access surfaces.	Access surfaces expose the kernel without becoming runtime authority.
D-011	Defines inactive candidate quarantine for memory, knowledge, learning, prompt, routing, and workflow proposals.	Proposed evolution remains inspectable until promoted by authority.

Architecture Viewpoints

11.1 Viewpoint Catalogue

Viewpoint	Primary concern	MADRE content
System context	Runtime boundary and environment	User, local device, filesystem, operating system, local models, optional external integrations, internal actions, and untrusted sources.
Logical	Responsibility separation	Kernel, Complex Agent lanes, delayed reasoning, scheduler, context/knowledge, local model runtime boundary, Internal Action Registry, policy, learning, audit, and recovery.
Runtime/process	Work lifecycle	User turn, triage, immediate answer, reasoning request, context bundle, scheduling, execution, verification, result delivery, and recovery.
Data/knowledge	Data ownership and provenance	Event records, context bundles, knowledge areas, private memory, source material, sensitivity, retention, correction, deletion, and learning candidates.
Security/trust	Boundary enforcement	Local domain, governed external boundary, external/risky domain, prompt injection, remote transfer, credential exposure, and action side effects.

Viewpoint	Primary concern	MADRE content
Deployment	Local operation	Ordinary device profile, local storage, model availability, backup, migration, diagnostics, and optional remote configuration.
Extension	Developer evolution	Model runtime bindings, action contracts, workflows, projections, context policies, and interface surfaces.
UX	User-visible control	Conversation, clarification, queue state, progress, authorization, failure, completion, correction, and rollback.

11.2 Viewpoint Design Focus

Viewpoint	Architecture focus	Authority boundary
System context	Local user, local device, CLI, local read-first console, local sources, and optional remote integrations.	Local authority remains explicit across every access surface.
Logical	Kernel-owned authority over triage, delayed work, context, models, bounded action use, candidates, policy, audit, and recovery.	Runtime authority is not delegated to interfaces, models, actions, or candidates.
Runtime/process	Governed lifecycle from local request to inspection and recovery.	Work transitions remain durable, auditable, and recoverable.
Data/knowledge	Governed context plus inactive candidate quarantine.	Source material, memory, knowledge, and learned candidates remain distinguishable.
Security/trust	Source-as-data, model-as-function, policy before action, and bounded internal action authority.	Action execution follows declared policy outcomes.

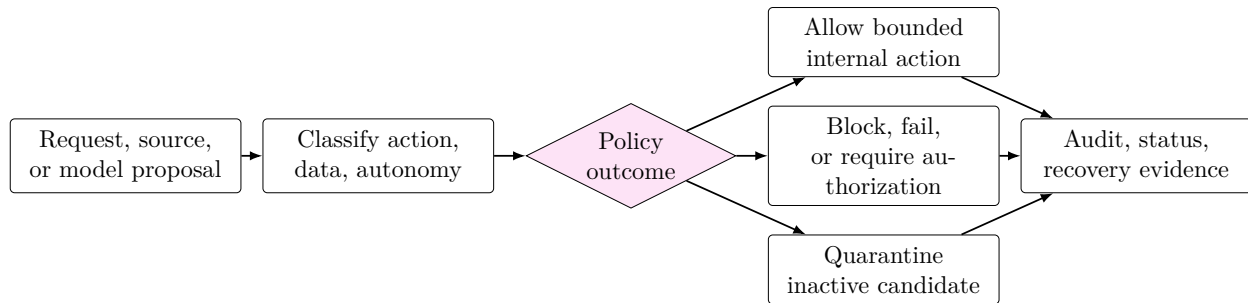
Viewpoint	Architecture focus	Authority boundary
Observability/recovery	Local status, audit, verification, candidate, and recovery evidence.	Read-first inspection must not become a privileged mutation path.
Extension	Runtime bindings, actions, workflows, and learning systems inherit the defined boundaries.	Extension preserves kernel authority and product traceability.

Runtime Governance

12.1 Runtime Authority Rule

Runtime authority belongs to the **MADRE** kernel and its policy boundaries. User turns, source text, model output, action results, local inspection, and candidate material can inform decisions, but they cannot authorize their own authority, promote themselves, mutate protected state, or widen permissions. Bounded internal action use is policy-gated, local access is read-first where it inspects state, and memory/knowledge/learning changes remain inactive candidates unless promoted by authority.

12.2 Policy And Action Gate



12.3 Policy Matrix

Action class			Data class		Autonomy level	Authorization	Audit/Recovery
Read	local	meta-data	Public	or low-sensitivity metadata	Observation	None by default	Log access when used for reasoning.
Draft text			Any	allowed context bundle	Draft creation	None unless sensitive scope is crossed	Store as candidate output, not active state.
Update local knowledge			Scoped	personal or domain knowledge	Bounded mutation	Required when provenance, confidence, or sensitivity is weak	Versioned record with correction and rollback.

Action class	Data class	Autonomy level	Authorization	Audit/Recovery
Invoke internal action	Small task-scoped non-sensitive input	Bounded internal action	Policy allow required	Requested action, input class, result, policy outcome, and recovery expectation recorded.
Contact remote service	Minimized and classified transfer set	External action	Explicit policy authorization	Transfer record, provider, purpose, and retention assumption recorded.
Modify credentials, identity, legal, financial, destructive, or executable state	Sensitive or critical data	Critical action	Strong human authorization	Block unless recovery or explicit irreversible authorization exists.

Knowledge updates, prompt mutation, routing mutation, workflow mutation, and learning lifecycle behavior follow the same authority rule: proposed changes begin as candidates and become active only through promotion boundaries.

12.4 Learning Lifecycle

Learning follows a controlled lifecycle: observation, candidate creation, evidence collection, evaluation, promotion, active monitoring, and rollback. Memories, knowledge entries, prompt changes, routing preferences, workflow improvements, and model adaptation candidates remain inactive unless the appropriate promotion rule validates them for a defined scope.

Threat Model

13.1 Threat Taxonomy

Threat	MADRE exposure	Primary control	Verification surface
Prompt injection	Untrusted text attempts to alter policy, action execution, or memory.	Source classification and instruction/data separation.	Adversarial context scenario.
Sensitive disclosure	Private context leaks to a model, action, or remote service.	Data classification, minimization, and remote-transfer authorization.	Disclosure review.
Excessive agency	Generated output causes over-broad action.	Autonomy levels and action-class policy.	Policy matrix review.
Unsafe action execution	Action invocation produces side effects outside task scope.	Policy-gated internal action boundary; remote, host-command, filesystem, and mutation actions require stronger authority.	Allowed-action and blocked-action scenarios.
Poisoned knowledge	Retrieved or extracted material corrupts subsequent context.	Provenance, quarantine, confidence, and correction path.	Candidate quarantine and source-correction scenario.
Inspection misuse	Local console is treated as authority or mutation path.	Read-first inspection boundary and shared policy model.	Local inspection negative scenario.
Supply-chain or provider dependency	Model, dependency, or service changes behavior or availability.	Replaceable runtime bindings and dependency policy.	Runtime substitution review.

Threat	MADRE exposure	Primary control	Verification surface
Remote transfer risk	Local data is sent outside the user's domain without classification, minimization, authorization, or audit.	Local-first default and remote-transfer policy boundary.	Remote-contact blocked scenario.
Unbounded consumption	Delayed work consumes excessive compute, storage, or cost.	Resource budgets, queue limits, and status inspection.	Resource benchmark.
Audit/recovery gaps	A result cannot be explained, corrected, invalidated, superseded, rolled back, or detached.	Durable audit observations and recovery semantics.	Recovery trace scenario.
Model output as false authority	Fluent generated text is treated as fact, policy, action permission, or memory.	Generated-output candidate handling and verification.	Unsupported-output scenario.

Quality Model

14.1 Quality Attribute Scenarios

Attribute	Scenario	Measure or evidence
Responsiveness	The foreground remains conversational while deeper work is queued.	Foreground latency and clear work status.
Recoverability	A failed or unsafe delayed task can be inspected and corrected.	Audit trail, status transition, and rollback/correction path.
Auditability	A result can be explained through context, model, action, policy, and verification records.	Complete outcome evidence chain.
Maintainability	A model runtime binding or action contract can be replaced without changing kernel authority.	Contract and substitution review.
Portability	A useful subset runs on ordinary local hardware.	Local profile and resource benchmark.
Security	Untrusted context cannot silently become instruction or policy.	Prompt-injection and data-boundary scenarios.
Resource discipline	Delayed work is bounded by cost, time, storage, and compute policies.	Queue and resource-use metrics.
Privacy	Sensitive or unrelated local material is excluded from context and remote transfer unless scoped and policy-allowed.	Context exclusion and remote-transfer scenarios.
Model substitutability	Equivalent behavior is preserved across replaceable model profiles.	Model substitution scenario.
Context isolation	One task's context does not silently bleed into another topic, user, model invocation, action invocation, or candidate.	Context scope and revocation review.
Explainability and inspectability	Users and developers can see status, policy outcome, context evidence, model/action evidence, failures, candidates, and recovery state.	Local read-first inspection scenario.

Attribute	Scenario	Measure or evidence
Testability	Architecture claims can be checked through behavior scenarios and negative paths.	Requirements and decision trace review.

UX, Observability, And Recovery

15.1 Interaction Contract

State	User-visible meaning
Immediate answer	The foreground has enough safe local context to answer.
Clarification needed	Intent, scope, risk, or context is insufficient.
Queued reasoning	Work has been recorded for delayed execution.
Running task	Delayed reasoning or action work is active.
Blocked by policy	A policy boundary prevents execution or promotion.
Authorization required	The action crosses a configured authority boundary.
Failed task	Work ended without acceptable result or recovery.
Completed task	Result is available with audit evidence.
Corrected or rolled back	A previous result, memory, or knowledge change has been revised or reverted.
Invalidated or superseded	Prior evidence, source, or result is no longer active because it was corrected, revoked, reclassified, or replaced.
Deleted or detached	Active generated material is removed from use where allowed while accountable audit history remains distinguishable.
Learning candidate	An improvement proposal exists but is not active.

15.2 Observability Surfaces

The local inspection surface must expose sessions, tasks, event records, context bundles, model invocations, internal action evidence, authorizations, failures, metrics, checkpoints, rollbacks, invalidations, supersessions, deleted/detached active material, knowledge candidates, learning candidates, and recovery actions. The inspection surface is not a privileged bypass; it is a read-first view over the same policy-governed runtime.

Evaluation And Benchmark Specification

MADRE defines an architectural evaluation surface for testing whether the runtime preserves local authority, governed context, bounded action use, candidate-only learning, auditability, and recoverable delayed reasoning.

16.1 Benchmark Families

Benchmark family	Purpose
Functional correctness	Confirm expected runtime behavior for foreground-lane continuity and delayed work.
Delayed reasoning quality	Evaluate evidence handling, decomposition, verification, and result delivery.
Foreground triage	Confirm unsupported evidence-heavy answers are clarified or delayed rather than fabricated.
Context isolation	Confirm unrelated or sensitive knowledge does not enter the wrong bundle.
Model substitution	Confirm replaceable model profiles preserve the same boundary behavior and expose failures.
Policy enforcement	Confirm action, data, autonomy, authorization, and recovery rules are enforced.
Bounded internal action	Confirm the bounded internal action is policy-gated, audited, and harmless.
Blocked unsafe actions	Confirm remote, host-command, filesystem, mutation, and critical-action attempts do not execute under bounded internal-action authority.
Candidate quarantine	Confirm memory, knowledge, learning, prompt, routing, and workflow proposals remain inactive and inspectable.
Read-first observability	Confirm CLI/status and local console inspection expose lifecycle, policy, context, model, action, candidate, and recovery evidence.

Benchmark family	Purpose
Recovery	Confirm failed or unsafe outputs can be inspected, corrected, deleted/detached, invalidated, superseded, or rolled back.
Resource use	Measure latency, storage, queue time, model cost, and local compute use.
Reproducibility	Preserve enough configuration, context, and event history to review outcomes.
Hallucination resistance	Confirm generated output does not become truth, knowledge, or action without verification or promotion.

Governance, Deployment, And Limits

17.1 Secure Engineering Principles

MADRE requires secure defaults for local credentials, dependency selection, governed external boundaries, secrets handling, reviewable release records, and vulnerability response. Security evidence must cover both conventional software risks and model-mediated risks such as prompt injection, poisoned retrieval, unsafe output handling, excessive agency, and remote disclosure.

17.2 Open-Source Governance

MADRE is released under GPL-3.0. The open-source governance model must preserve license clarity, dependency review, contribution rules, security disclosure, release integrity, and community responsibility. Openness is part of the project value only when users and developers can inspect, adapt, and verify the runtime without surrendering private data to a platform owner.

17.3 Deployment And Operational Assumptions

The baseline deployment target is an ordinary local device with local storage, local configuration, optional local model execution, and optional remote integrations governed by policy. The operational profile includes storage paths, model setup assumptions, backup and restore, migration, diagnostics, log retention, offline behavior, and upgrade or rollback expectations.

Deployment assumptions remain conceptual at this level of specification. The product architecture defines authority, locality, inspection, recovery, and model-runtime and action boundaries without reducing deployment to a single operational shape.

17.4 Risks, Limitations, And Excluded Functions

MADRE limits active executable behavior by authority class. Workflow authoring, self-learning, speech interaction, graphical product surfaces, distributed inference, organization-scale deployment, and model fine-tuning remain governed by the same local-first, policy-governed, model-agnostic, auditable, and recoverable runtime boundary as the rest of the product.

Requirements Catalogue

ID	Requirement summary	Decision trace
FR-001	Preserve foreground-lane conversation while delaying work that requires evidence, verification, internal actions, or stronger context.	D-001, D-004.
FR-002	Record delayed reasoning work durably with inspectable status and recovery path.	D-001, D-003, D-008.
FR-003	Construct context bundles from sourced, classified, minimized, scoped, provenanced, and revocable information.	D-001, D-005, D-007.
FR-004	Invoke models through replaceable model invocation contracts.	D-001, D-006.
FR-005	Expose executable behavior only as declared, bounded, auditable internal actions or explicitly authorized external actions.	D-007, D-009.
FR-006	Keep memory, knowledge, prompt, routing, workflow, and learning proposals inactive unless promotion boundaries validate them.	D-011.
FR-007	Expose local request entry and read-first inspection.	D-010.
FR-008	Represent recovery outcomes explicitly.	D-003, D-008.
SEC-001	Treat prompt/source/model text as data or candidate output unless classified and validated by an authority boundary.	D-004, D-005, D-007.
OPS-001	Keep local access surfaces governed by runtime authority.	D-010.

Traceability Matrix

Requirement	Concern	Architecture element	Evidence family
FR-001	Responsiveness and user control	Complex Agent foreground lane and delayed reasoning split	Interaction and latency scenarios.
FR-002	Recoverability and audit	Local durable work records	Recovery and status scenarios.
FR-003	Privacy and correctness	Governed context bundle	Context isolation scenarios.
FR-004	Provider independence	Local model runtime boundary and model invocation action	Runtime substitution scenarios.
FR-005	Action safety	Internal Action Registry, action contract, and external action policy	Allowed-action and blocked-action scenarios.
FR-006	Controlled evolution	Candidate quarantine	Candidate-only learning review.
FR-007	Local operability	CLI plus read-first local console	Local access and inspection scenario.
FR-008	Recoverability	Audit history and recovery semantics	Recovery trace.

Threat Catalogue

Threat family	Primary boundary	Initial response
Prompt and context injection	Context bundle construction	Separate source material from instruction and policy.
Sensitive disclosure	Data classification and remote transfer	Minimize, redact, authorize, and audit disclosures.
Unsafe action execution	Action and workflow contracts	Require action-class policy, bounded internal-action scope, and recovery expectation.
Knowledge poisoning	Knowledge extraction and promotion	Quarantine, provenance, confidence, correction, and rollback.
Inspection bypass	Local console and status surfaces	Read-first inspection with no privileged mutation path.
Model authority drift	Generated output boundary	Treat generated text as candidate output, not authority.
Provider dependency	Local model runtime boundary	Runtime substitution and local-first default.
Unbounded consumption	Scheduler and resource controls	Queue, compute, storage, and cost limits.

Benchmark Catalogue

Benchmark	Observed property	Evidence
Foreground response	Conversational continuity under delayed work	Latency and interaction-state record.
Delayed work durability	Work survives interruption and exposes status	Event lifecycle record.
Context isolation	Only validated knowledge enters a task context	Context bundle inspection.
Policy enforcement	Disallowed action remains blocked or requires authorization	Policy decision record.
Bounded internal action	Bounded internal action remains local, policy-gated, and auditable	Action trace.
Candidate quarantine	Learning and knowledge proposals remain inactive	Candidate inspection record.
Read-first inspection	Local console exposes status, audit, policy, context, model, action, candidate, and recovery evidence	Inspection trace.
Recovery	Failed or unsafe result can be corrected or rolled back	Recovery trace.
Runtime substitution	Model invocation contract survives backend replacement	Runtime boundary comparison.

Conceptual Module And Agent Sketch

This appendix is a conceptual sketch. It is not normative class design. It does not authorize implementation of classes, attributes, methods, inheritance, package layout, schema, persistence, queue topology, prompt format, API shape, or runtime wiring. Names in this appendix are candidate vocabulary only where they are also promoted through the specification and decision records.

E.1 Module Ecosystem Sketch

Sketch element	Conceptual meaning
MADREKernel	Coordinates module registration, routing, global policy root, resource limits, scheduling, persistence coordination, audit root, Internal Action Registry, and lifecycle. It is not itself a reasoning module unless a later architecture decision explicitly changes that.
ReasoningModule	Governed unit for a bounded reasoning domain. It owns scope, authority profile, module-local policy envelope, queues, workflows, audit stream, knowledge/cache boundaries, assignable agents, and a system agent.
MADRECoreModule	Central reasoning module for incoming interaction, conversation continuity, foreground-lane response, clarification, dynamic reasoning plan creation, delegation, and delayed-work decisions.
ResourceOptimizationModule	Possible future module that observes resource, latency, model, and workflow evidence and proposes configuration or routing candidates without directly changing kernel policy or protected state.

E.2 Agent, Workflow, Routine, And Plan Sketch

Sketch element	Conceptual meaning
MADREAgent	Model-agnostic runtime actor inside a module. It executes tasks through routines and selects model invocation actions or internal actions through strategy. It is not a model runtime binding.
SimpleAgent	Direct execution depth profile: one routine path, one primary invocation or action path, one task response.
TwinAgent	Double-checking depth profile: primary result plus review through a second instance, model, prompt, routine, or process.
ComplexAgent	Temporal supervision depth profile: primary work, recent-work supervision, and delayed audit/outcome observation that may produce improvement candidates. It does not gain extra authority by being complex.
AgenticWorkflow	Module-level orchestration that coordinates one or more agents and routines, possibly in parallel, under module authority and kernel policy.
AgenticRoutine	Agent-level recipe for a habitual task or action, such as reading allowed input, drafting, reviewing, invoking a bounded internal action, or proposing a candidate.
ReasoningPlan	Dynamic request-derived plan that may select workflows, decompose tasks, assign agents, and define audit or recovery expectations. It is not a reusable workflow by itself.

E.3 Candidate And Artifact Sketch

Sketch element	Conceptual meaning
Candidate	Inactive proposal for promotion, persistence, behavior change, memory, knowledge, routine, workflow, routing, prompt, policy, model strategy, response acceptance, or artifact materialization. A candidate is not active state.
Artifact	Materialized output such as a file, document, code patch, report, image, dataset, PDF, snapshot, or final buffer/document. An ordinary response is not an artifact unless it is materialized.
AuditEvent	Typed persistent evidence that can correlate request, plan, workflow, task, routine, agent, module, action, policy, artifact, candidate, and recovery outcome. It is not merely a raw log string.

Glossary

Term	Definition
Internal action	A typed runtime-defined executable function available to modules through the Internal Action Registry. An internal action declares inputs, outputs, side-effect class, policy requirements, audit evidence, and recovery expectations. It is not an exposed network integration surface.
Action contract	The declaration that defines an internal action's input schema, output schema, side effects, authorization requirements, audit obligations, and recovery expectations.
Model invocation action	A specialized internal action that invokes a local or policy-authorized model runtime. It receives governed context, model strategy, action policy, decoding configuration, and returns generated candidate material plus inference evidence.
Complex Agent	A module-local runtime actor that may coordinate foreground, reasoning, and reflection lanes. It can answer, defer, reason, verify, and propose learning candidates, but it does not gain authority beyond module and kernel policy.
Local model runtime boundary	The controlled boundary through which MADRE reaches local or policy-authorized model execution. It includes model profiles, quantization choices, device/resource policy, session/cache policy, inference traces, and generated candidate handling.
Reasoning query	A recorded need for delayed analysis, evidence, internal action invocation, or verification beyond the foreground answer.
Knowledge area	A scoped collection of source-backed knowledge with provenance, sensitivity, correction, and deletion rules.
Model profile	A declared model execution profile with purpose, limits, cost, context behavior, and policy constraints.
Read-first console	A local inspection surface that exposes status and audit evidence without becoming a privileged mutation path.
Inactive candidate	Proposed memory, knowledge, learning, prompt, routing, or workflow material that is inspectable but not active runtime state.
Audit event	A durable observation that records what happened, under which context, model invocation, action invocation, policy, and verification boundary.

Product Boundary Statement

MADRE is a local-first, model-agnostic runtime kernel for governed agentic systems. Product behavior is defined by the architecture, requirements, safety boundaries, traceability, and validation criteria in this specification.